POSITION PAPER · PROCESS SAFETY · AI SYSTEMS

Applying HAZOP to AI Systems

What Process Engineering Knows About Failure That AI Safety Doesn't

Jonathan Horvat

2026 · PUBLIC DOMAIN · NO RIGHTS RESERVED

ABSTRACT

Process engineering has spent sixty years developing formal methodologies for identifying and managing failure modes in complex systems where errors have physical consequences. The most widely used of these — Hazard and Operability Study, or HAZOP — is a structured, node-by-node analysis that asks what happens when each component of a system deviates from its design intent. The methodology produces a formal register with specified causes, consequences, safeguards, and required actions. That register has legal standing. It gets reviewed. When the system changes, it gets updated.

AI systems deployed in consequential contexts — physical AI, medical AI, infrastructure AI, any AI where the output affects the physical world — have no equivalent. Failure modes are discovered in production. Responses are improvised. Nobody decided how the system fails. This paper argues that HAZOP applies directly to AI systems, demonstrates the methodology on a specific architecture, and proposes a standard for what an AI HAZOP should produce.

> *This paper is a companion to:* Horvat, J. (2026). *Intelligence at the Edge of the Cloud: A Local Architecture for the AI Era.* Public domain.

SECTION I

Two Industries, One Problem

In process engineering, the question of how a system fails is not left to chance or discovered in production. It is decided — deliberately, formally, at the design stage — before a single valve is specified or a pipe is routed. A cooling water valve fails open because losing cooling is the catastrophic outcome. A fuel supply valve fails closed because uncontrolled fuel flow is the catastrophic outcome. The failure mode is a design decision, made by a process engineer, documented in the process design package, and enforced in hardware. The valve does not decide how to fail. The system designer decides.

This principle — that failure modes must be specified, not discovered — is the foundation of process

safety. It emerged from catastrophic industrial accidents in the mid-twentieth century, most notably Flixborough in 1974, where a temporary pipe bypass failed under pressure, killing 28 people and destroying the plant. The accident investigation revealed that nobody had formally analyzed what would happen if the bypass failed. The consequences were not unknown — they were simply unasked.

AI systems in 2026 are in approximately the same position that chemical plants were in 1963, when ICI first developed the operability study methodology. Complex. Consequential. Widely deployed. And almost entirely without formal failure mode analysis. When a cloud-dependent AI system loses connectivity, what happens? When the model receives data it was not trained on? When latency exceeds acceptable bounds? In most deployments, nobody decided. The system does whatever it does, and the response is improvised.

> *Process engineering solved this problem sixty years ago. The methodology transfers directly. The only thing missing is the decision to apply it.*

→ [1, 2]

SECTION II

HAZOP Fundamentals

A Hazard and Operability Study is a structured examination of a system, conducted by a multidisciplinary team, that systematically identifies all credible deviations from design intent and their consequences. It produces a formal register — the HAZOP register — that documents every identified hazard, its causes, consequences, existing safeguards, and required actions. The register is the deliverable. It has legal standing in regulated industries. It is updated when the system changes.

The methodology proceeds in three steps. First, the system is divided into nodes — discrete sections, each with a defined design intent. In process piping, nodes are typically pipe sections or vessels. In AI systems, nodes are functional components. Second, parameters relevant to each node are identified — in process systems these are physical quantities like flow, pressure, temperature, and composition. In AI systems the equivalents are data, signal, weights, context, connectivity, latency, and confidence. Third, guide words are applied systematically to each parameter to generate deviations. The guide words are the engine of the methodology.

HAZOP GUIDE WORDS — STANDARD DEFINITIONS WITH PROCESS AND AI EQUIVALENTS

GUIDE WORD	DEFINITION	PROCESS EXAMPLE	AI EQUIVALENT
No / None	Complete negation of design intent	No flow in a transfer line	No sync received · No data to model · No response generated
More	Quantitative increase beyond design	High pressure in vessel	More queries than capacity · More data than expected · Higher confidence than warranted
Less	Quantitative decrease below design	Low flow in cooling line	Less training data · Lower sync frequency · Reduced coverage

As Well As	Qualitative addition to design intent	Contamination in process stream	Poisoned weights alongside valid ones · Personal data in world model sync
Part Of	Qualitative reduction of design intent	Partial composition only	Incomplete sync · Partial adapter update · Truncated response
Reverse	Logical opposite of design intent	Reverse flow in pipe	Model transmits instead of receives · Adapter trains on wrong signal
Other Than	Complete substitution of design intent	Wrong chemical in line	Wrong model version synced · Query routed to wrong expert · Wrong user context applied

Not every guide word applies to every parameter at every node. The methodology does not require exhaustive application — it requires systematic application of all *credible* deviations. A combination that produces no meaningful deviation is simply not recorded. The skill is in identifying which combinations matter, which requires domain knowledge of both the methodology and the system being analyzed.

→ [1, 3]

SECTION III

Translating HAZOP to AI

The translation from process parameters to AI parameters is direct. Each physical quantity in a process system has a functional equivalent in an AI system. The mapping is not metaphorical — it is structural. Both systems have inputs, throughputs, transformations, and outputs. Both have design intents that can be deviated from. Both have consequences when deviations occur.

PARAMETER TRANSLATION — PROCESS ENGINEERING → AI SYSTEM EQUIVALENTS

PROCESS PARAMETER	AI EQUIVALENT	DESIGN INTENT EXAMPLE
Flow	Data / Signal / Query rate	Queries arrive at expected rate within model capacity
Pressure	Load / Demand / Inference queue depth	Inference queue depth remains within latency budget
Temperature	Latency / Thermal state of device	Response latency within acceptable bounds; device within thermal envelope
Composition	Model weights / Version / Data distribution	Weights are current, validated, and match expected architecture version
Level	Memory / Storage / Context window occupancy	Context window occupancy within model capacity; storage sufficient for adapters
Connectivity	Network / Sync channel availability	Sync channel available for scheduled weight updates
Reaction	Adapter training / Model update	Adapter weights update through behavioral observation within expected bounds
Signal	Confidence score / Coverage score / Output certainty	Coverage score accurately reflects actual domain coverage

The node structure for an AI system follows the same logic as process piping — each node is a functional section where the design intent can be stated clearly enough to generate meaningful deviations. For the broadcast sync architecture analyzed in this paper, six nodes are defined: the sync channel, the world model, the personal adapters, the interpreter head, the coverage score mechanism, and the user interface. Each has a clear design intent. Each has parameters that can deviate. Each deviation has consequences that can be specified.

SECTION IV

The HAZOP Register Applied

The following register applies the HAZOP methodology to the broadcast sync architecture described in *Intelligence at the Edge of the Cloud*. The register follows the standard nine-column format used in process industries: Node, Parameter, Guide Word, Deviation, Cause, Consequence, Existing Safeguards, Action Required, and Responsibility. Severity is indicated as H (High — safety or major function impact), M (Medium — degraded operation), or L (Low — minor impact, self-correcting).

Only credible deviations are recorded. Combinations that produce no meaningful consequence are omitted. The register is not exhaustive — a full production HAZOP would require a multi-day workshop with domain specialists. This register demonstrates the methodology and identifies the most significant hazards.

AI HAZOP REGISTER — BROADCAST SYNC ARCHITECTURE H = HIGH · M = MEDIUM · L = LOW SEVERITY

NODE	PARAMETER	GUIDE WORD	DEVIATION	CAUSE	CONSEQUENCE	EXISTING SAFEGUARDS	ACTION REQUIRED	RES
NODE 1 · SYNC CHANNEL — DESIGN INTENT: WORLD MODEL WEIGHTS BROADCAST PERIODICALLY FROM SERVER TO DEVICE; DE RETAINS RELEVANT EXPERTS LOCALLY								
N1	Connectivity	No	No sync received — extended offline period	Network unavailable; server offline; device in coverage gap	M World model knowledge becomes stale. Coverage score degrades over time. System continues to function on existing weights.	Coverage score timestamps experts; DH component of IpMW degrades gracefully. User is informed of sync date.	Define maximum acceptable staleness threshold. Specify behavior when threshold exceeded — should system reduce confidence scores automatically?	Arc Des
N1	Composition	As Well As	Malicious weights alongside valid weights in broadcast	Compromised sync server; supply chain attack on model weights; adversarial weight injection	H Corrupted world model behavior. Potential for targeted misinformation, capability degradation, or covert data exfiltration via model behavior.	Human-gated publication — weights require deliberate authorization before broadcast. No automatic write path from training to sync server.	Implement cryptographic signing of weight packages. Device verifies signature before integrating sync. Define process for emergency weight revocation.	Sec / C

N1	Composition	Part Of	Incomplete sync — partial expert package received	Network interruption mid-sync; storage limit reached on device; timeout during large update	L Expert partially updated. Router may direct queries to expert with inconsistent weight version. Minor capability degradation in affected domain.	Delta-patching protocol; version tracking per expert. Incomplete sync detected by checksum mismatch.	Specify atomic sync behavior — either full expert package integrates or none does. No partial expert state permitted.	Arc Des
N1	Flow	More	Sync frequency higher than intended — excessive update rate	Server misconfiguration; CDN cache invalidation loop; client retry storm	L Excessive bandwidth consumption. Battery drain from repeated sync activity. No safety consequence.	Client-side rate limiting on sync requests. Exponential backoff on retry.	Define maximum sync frequency. Implement server-side rate limit per device cohort.	Pla Eng
NODE 2 · WORLD MODEL — DESIGN INTENT: SPARSE EXPERT MODEL RESPONDS TO QUERIES FROM AUGMENTED PERSONAL MODEL PULL-ONLY; NO PERSONAL DATA ACCESS; NO TRANSMISSION								
N2	Signal	Reverse	World model initiates communication rather than responding to queries	Implementation defect; compromised runtime; malicious weight injection enabling active behavior	H Fundamental breach of privacy architecture. World model could exfiltrate personal context if it can initiate outbound communication.	Architectural constraint — world model has no transmission interface by design. OS-level network policy blocks outbound requests from world model process.	Enforce OS-level network isolation for world model process. Include in security audit scope. Add runtime assertion that world model never opens a network socket.	Sec
N2	Flow	No	World model unresponsive — no output to interpreter head	Model crash; out-of-memory; thermal throttling; corrupt expert weights	M Augmented personal model cannot synthesize world knowledge. System falls back to personal adapters only. Response quality degrades to base adapter capability.	Coverage score detects world model unavailability. User informed of degraded state. Personal adapters continue functioning.	Define explicit fallback behavior when world model is unavailable. Specify whether system should respond from personal adapters alone or decline the query.	Arc Des
N2	Composition	Other Than	Wrong expert activated for query domain	Router miscalibration; query domain ambiguous; expert boundary overlap in training	M Response draws on incorrect domain knowledge. Coverage score may not detect mismatch if router is confidently wrong. Plausible but incorrect output.	Coverage score includes router activation confidence. Low confidence triggers uncertainty signal to user.	Add domain classification validation layer between router and expert activation. Specify minimum router confidence threshold below which coverage score is automatically reduced.	ML Des
NODE 3 · PERSONAL ADAPTERS — DESIGN INTENT: LOW-RANK ADAPTER WEIGHTS ENCODE USER PATTERNS THROUGH BEHAVIORA OBSERVATION ONLY; NEVER TRANSMITTED; UPDATED INCREMENTALLY ON-DEVICE								
N3	Flow	Reverse	Personal adapter weights transmitted off-device	Implementation defect; malicious application accessing adapter via OS vulnerability; secure enclave	H Complete privacy breach. Personal adapter contains behavioral fingerprint of	Hardware secure enclave isolates adapter weights. No API exposes raw adapter weights externally. OS-level access controls.	Include adapter transmission prevention in security audit. Add runtime assertion monitoring for	Sec

				breach	user. Transmission exposes identity, patterns, and potentially sensitive context.		any adapter data leaving enclave boundary. Third-party security verification.	
N3	Reaction	More	Adapter over-trains — excessive adaptation to recent interaction pattern	Unusual interaction session; adversarial prompt sequence designed to manipulate adapter; rapid behavior change in user	M Adapter over-fits to recent session. Prior user context underweighted. System behaves inconsistently across sessions. Potential for adversarial manipulation of personal model behavior.	Incremental update rate limiting. Exponential decay on adapter update magnitude.	Define maximum adapter update magnitude per session. Specify drift detection mechanism — if adapter weights change beyond threshold in single session, flag for review and limit update rate.	ML Des
N3	Reaction	No	Adapter fails to update — no behavioral learning occurs	Observation signal below update threshold; enclave compute unavailable; adapter storage full	L Personalization delta (PΔ) remains at zero. System functions as base model only. No safety consequence — base model remains fully functional.	Coverage score reports personalization state. User can be informed if adapter calibration is not progressing.	Define minimum interaction rate required for adapter update. Specify storage management policy for adapter weights.	Pla Eng
NODE 4 · INTERPRETER HEAD — DESIGN INTENT: FIXED NON-TRAINABLE CROSS-ATTENTION MODULE TRANSLATES BETWEEN WO MODEL AND PERSONAL MODEL EMBEDDING SPACES; FROZEN AT COMPILE TIME								
N4	Composition	Other Than	Interpreter head version mismatch — compiled against different base model version than currently installed	Base model sync updates embedding space geometry; interpreter head not recompiled; version management failure	M Cross-attention keys and queries misaligned across embedding spaces. Synthesis quality degrades. Coverage score may not detect geometric mismatch. Subtle but pervasive response quality degradation.	Version locking between base model and interpreter head. Sync rejected if interpreter head version does not match base model version.	Treat interpreter head as tightly coupled to base model version. Any base model sync must trigger interpreter head recompilation or sync rejection. Define versioning protocol.	Arc Des
N4	Signal	No	Interpreter head produces no output — cross-attention fails	Null world model output passed to cross-attention; dimension mismatch; numerical instability	L Synthesis step fails. System falls back to personal model response without world model integration. Response quality degrades but system does not crash.	Null output detection in synthesis pipeline. Fallback to personal model response with coverage score indicating world model unavailable.	Implement explicit null-output handler in interpreter head. Specify fallback behavior. Add to integration test suite.	ML Des
NODE 5 · COVERAGE SCORE — DESIGN INTENT: WEIGHTED COMBINATION OF ROUTER CONFIDENCE, EXPERT FRESHNESS, AND DOMAIN RELEVANCE PRODUCES ACCURATE UNCERTAINTY SIGNAL SURFACED TO USER								

N5	Signal	More	Coverage score overestimates actual coverage — confident signal in low-coverage domain	Router overconfidence; calibration drift; distribution shift in query domain not detected by router	H User receives confident response in domain where model is unreliable. Hallucination presented as high-confidence output. Degrades DH component of IpMW to near zero. Most dangerous failure mode in the architecture.	None beyond router confidence — this is the primary open risk in the architecture.	Implement external calibration validation. Periodically test coverage score accuracy against known-answer queries in boundary domains. Alert if DH drops below threshold. This is the highest-priority action item in this register.	ML Des QA
N5	Signal	Less	Coverage score underestimates actual coverage — false modesty in well-covered domain	Router underconfidence; conservative calibration; stale freshness timestamp despite good expert quality	M User unnecessarily warned of uncertainty in domain where model is reliable. Erodes trust in the coverage score signal. System appears less capable than it is.	User experience degradation only. No safety consequence.	Include underconfidence detection in calibration testing. Tune freshness decay function to avoid excessive staleness penalty for high-quality experts.	ML Des
NODE 6 · USER INTERFACE — DESIGN INTENT: QUERY RECEIVED FROM USER; RESPONSE AND COVERAGE SCORE DELIVERED TO USER; NO PERSONAL CONTEXT TRANSMITTED EXTERNALLY								
N6	Flow	As Well As	Personal context transmitted alongside query to external service	Third-party application integration that logs queries; UI layer sending context to analytics endpoint; developer error in application built on architecture	H Privacy architecture bypassed at the application layer. Personal context reaches external infrastructure despite architectural protections in the model layer.	Architecture prevents model-layer transmission. Application layer is outside the core trust boundary.	Publish clear application developer guidelines prohibiting query logging with personal context. Define privacy API contract. Consider OS-level enforcement of query privacy for applications using the architecture.	Pla / P
N6	Signal	No	Coverage score not surfaced to user — uncertainty signal suppressed	Application developer chooses not to display coverage score; UI implementation omits uncertainty display	M User receives confident-appearing response with no uncertainty signal. Degradation honesty benefit of architecture lost at presentation layer. Effectively equivalent to architectures without coverage score.	None — application layer decision.	Specify minimum coverage score display requirement in API contract. Applications that suppress coverage score below threshold should surface a generic uncertainty indicator.	Pla / P

The register contains 14 credible deviations across 6 nodes, with severity ratings of High (5), Medium (7),

and Low (2). The most significant finding — rated High severity with no existing safeguard — is the coverage score overconfidence scenario in Node 5. This is the architecture's primary open risk: a router that confidently directs queries to an unsuitable expert produces a confident-appearing response in a domain where the model is unreliable, and the coverage score fails to warn the user. This is not a flaw unique to this architecture. It is the central unsolved problem in AI uncertainty quantification, surfaced explicitly by the HAZOP methodology.

→ [4, 5]

SECTION V

What the Register Reveals

The HAZOP register confirms that the broadcast sync architecture was designed with conservative failure in mind. The majority of identified deviations have existing safeguards — the architecture degrades gracefully rather than failing catastrophically. This is consistent with a design that treats offline operation and uncertainty as first-class properties rather than edge cases.

But the register also surfaces three genuine open questions that the architecture does not fully answer.

01 **Coverage score overconfidence has no architectural safeguard**

The Node 5 High severity finding — router overconfidence producing a false coverage signal — is the primary open risk. The architecture relies on the router being well-calibrated, but router calibration drift is a known failure mode in sparse MoE systems. An external calibration validation loop, periodically testing coverage score accuracy against known-answer queries, is the required action but is not yet specified in the architecture.

02 **Failure mode for world model unavailability is not formally specified**

When the world model is unresponsive (Node 2, No Flow), the system falls back to personal adapters only. This is the right behavior — conservative degradation rather than failure. But the specific behavior is not formally specified as a design decision. Should the system respond from adapters alone, or decline queries in domains that require world model knowledge? The answer depends on the deployment context and must be decided at the process design level, not the implementation level.

03 **Application layer privacy enforcement is outside the trust boundary**

The Node 6 High severity finding — personal context transmitted via application layer — exposes a gap between the architecture's strong model-layer privacy guarantee and the weaker application-layer enforcement. The architecture cannot prevent a developer from logging queries with personal context. This is the process engineering equivalent of a plant with excellent internal safety but no control over what happens after the product leaves the gate. A platform-level privacy API contract is required.

→ [4, 5, 6]

SECTION VI

The Failure Mode Decision

The most important principle in process safety is not the HAZOP itself. It is the principle that underlies it: failure modes must be decided, not discovered. In process engineering, how a system fails is a design decision made by a qualified engineer at the earliest stage of design, before hardware is specified. The valve does not decide how to fail. The process engineer decides, based on which failure mode is safest for that specific point in the system, at that specific operating condition, in that specific process context.

This decision is then documented, reviewed, challenged by a multidisciplinary team, and enforced in hardware. If the process changes, the failure mode analysis is revisited. There is always an answer to the question: who decided how this fails, and when?

AI systems have no equivalent. The question of how an AI system fails — what it does when connectivity is lost, when it receives out-of-distribution input, when its confidence is wrong, when the user acts on an incorrect response — is almost never asked at the design stage. It is answered in production, after something goes wrong, by whoever happens to be responsible at the time.

This is not a criticism of AI developers. It reflects the absence of a methodology. Process engineering did not develop conservative failure mode design through intuition — it developed it through catastrophic accidents and the formal frameworks those accidents inspired. The AI industry has not yet had its Flixborough. As AI systems become more physically consequential — controlling vehicles, managing medical devices, operating infrastructure — it will.

> *The question is not whether AI systems will fail. They will. The question is whether someone decided how they fail, before they did.*

→ [1, 2, 6]

SECTION VII

A Proposed Standard

The following outlines what an AI HAZOP standard should require for consequential deployments. Consequential is defined as any AI system where the output directly influences a physical, medical, financial, or safety-critical decision.

PROPOSED AI HAZOP STANDARD ELEMENTS

WHEN	Conducted before initial deployment of any consequential AI system. Revalidated when the system architecture changes materially — new model version, new deployment context, new integration point. An as-built HAZOP conducted before any production launch.
WHO	Multidisciplinary team including: ML systems engineer, domain expert for the deployment context, safety engineer or process engineer familiar with the methodology, end-user representative, and independent HAZOP facilitator. Minimum five participants. Maximum team size twelve.

NODES	System divided into functional nodes, each with a stated design intent. Minimum nodes: data input, model inference, output generation, user interface, external integrations, update/sync mechanism. Additional nodes for domain-specific components.
PARAMETERS	Data, signal, weights/version, context, connectivity, latency, confidence, and any domain-specific parameters. Guide words applied systematically to each credible parameter at each node.
REGISTER	Nine-column register in standard format: Node, Parameter, Guide Word, Deviation, Cause, Consequence, Existing Safeguards, Action Required, Responsibility. Severity rating required for each row. Register is a controlled document — versioned, dated, and updated when system changes.
FAILURE MODES	For each High severity deviation with inadequate safeguards, the team must specify the intended failure mode explicitly: fail-safe (conservative degradation), fail-operational (continue at reduced capability), or fail-stop (cease operation). The decision must be documented and the rationale recorded.
OWNERSHIP	Every action item in the register must have a named responsible party and a completion date. The register is not complete until all High severity items either have adequate safeguards or have a documented decision to accept the risk with named acceptance authority.
REVIEW CYCLE	Annual revalidation for deployed systems. Immediate revalidation triggered by: any High severity finding from production incidents, any material architecture change, any change in deployment context that introduces new failure modes.

This standard does not require new technology. It requires applying a methodology that has been refined over sixty years to a new class of system. The tools — guide words, node analysis, register format — are the same. The parameters are different. The discipline required is the same.

Process engineering did not invent the concept of conservative failure mode design. It systematized it. The AI industry needs to do the same, and it does not need to wait for a catastrophic accident to begin.

→ [1, 2, 3, 7]

REFERENCES

[1] IEC 61882:2016. *Hazard and Operability Studies (HAZOP Studies) — Application Guide.* International Electrotechnical Commission. — The international standard governing HAZOP methodology.

[2] Kletz, T. A. (1999). *HAZOP and HAZAN: Identifying and Assessing Process Industry Hazards.* 4th ed. Institution of Chemical Engineers. — Foundational text on HAZOP methodology by the engineer who systematized it.

[3] Crawley, F. & Tyler, B. (2015). *HAZOP: Guide to Best Practice.* 3rd ed. Elsevier. — Standard reference for HAZOP register format and node analysis methodology.

[4] Horvat, J. (2026). *Intelligence at the Edge of the Cloud: A Local Architecture for the AI Era.* Public domain. — Primary architecture paper; defines the six nodes analyzed in this register.

[5] Guo, C. et al. (2017). *On Calibration of Modern Neural Networks.* ICML 2017. arXiv:1706.04599. — Grounds the coverage score overconfidence finding in established calibration literature.

[6] Amodei, D. et al. (2016). *Concrete Problems in AI Safety.* arXiv:1606.06565. — AI safety framework; HAZOP addresses several of the identified problem categories through formal process engineering methodology.

[7] Health and Safety Executive (HSE) (1999). *A Guide to the Control of Major Accident Hazards Regulations.* — Regulatory

framework that established HAZOP as legally required methodology for major hazard facilities; proposed model for AI regulation.

Public domain. No rights reserved. Build freely.

JONATHAN HORVAT · 2026